

Recursión de cola en Python: teoría, patrones y práctica

Mario Alejandro Bravo Ortiz

Agenda de la clase

01

Fundamentos de recursión

Repaso de conceptos básicos y análisis de la pila de llamadas

02

Recursión de cola

Definición formal, características y comparación con recursión tradicional

03

Python y TCO

Limitaciones del intérprete y estrategias de optimización

04

Patrones y transformaciones

Conversión de recursión tradicional a recursión de cola

05

Práctica aplicada

Ejercicios graduados con soluciones completas

¿Qué es la recursión?

Definición formal

Una función recursiva es aquella que se invoca a sí misma, directa o indirectamente, para resolver un problema dividiéndolo en subproblemas más pequeños de la misma naturaleza.

Componentes esenciales:

- Caso base: condición de terminación
- Caso recursivo: llamada a sí misma
- Reducción del problema hacia el caso base

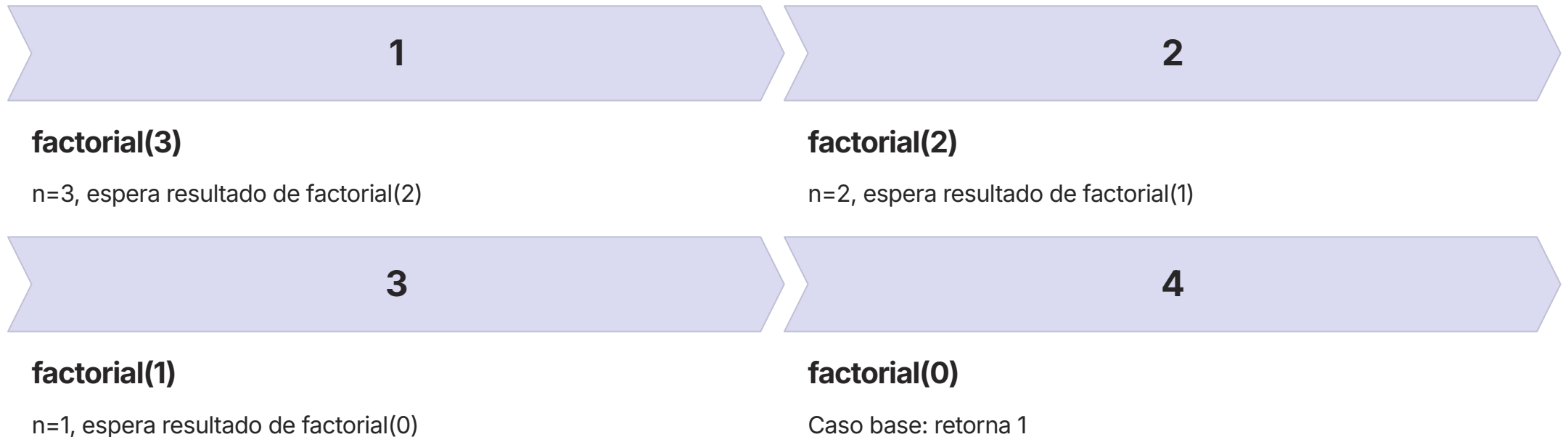
Ejemplo clásico: factorial

```
def factorial(n):  
    # Caso base  
    if n == 0:  
        return 1  
    # Caso recursivo  
    return n * factorial(n - 1)  
  
# Uso  
print(factorial(5)) # 120
```

Cada llamada debe acercarse al caso base para evitar recursión infinita.

La pila de llamadas (call stack)

Cuando una función se llama a sí misma, Python almacena cada invocación en la **pila de llamadas**. Cada marco de pila contiene variables locales, parámetros y la dirección de retorno.



La pila crece con cada llamada y se desapila al retornar valores. En Python, el límite predeterminado es `sys.getrecursionlimit()`, típicamente 1000.

Problema: límite de recursión en Python

```
import sys

def factorial_ingenuo(n):
    if n == 0:
        return 1
    return n * factorial_ingenuo(n - 1)

print(sys.getrecursionlimit()) # 1000

try:
    resultado = factorial_ingenuo(1500)
except RecursionError as e:
    print(f"Error: {e}")
```

Salida esperada:

```
1000
Error: maximum recursion depth exceeded
```

¿Por qué ocurre esto?

Cada llamada recursiva consume espacio en la pila. Con 1500 llamadas anidadas, Python alcanza su límite de seguridad y lanza `RecursionError`.

Aunque podemos aumentar el límite con `sys.setrecursionlimit()`, esto no resuelve el problema fundamental: **consumo de memoria**.

Recursión de cola: definición



Característica clave

La llamada recursiva es la **última operación** que realiza la función antes de retornar. No hay cálculos pendientes después de la llamada.



Ventaja teórica

En lenguajes con **Tail Call Optimization (TCO)**, el compilador/intérprete puede reutilizar el marco de pila actual, evitando crecimiento de la pila.



Limitación en Python

Python NO implementa TCO. Las funciones con recursión de cola consumen pila igual que la recursión tradicional. Es una decisión de diseño del lenguaje.



Ejemplo comparativo

Recursión tradicional

```
def suma_trad(n):  
    if n == 0:  
        return 0  
    # Cálculo DESPUÉS de la llamada  
    return n + suma_trad(n - 1)  
  
print(suma_trad(5)) # 15
```

La suma `n +` ocurre **después** de que `suma_trad(n-1)` retorna. Hay operaciones pendientes.

Recursión de cola

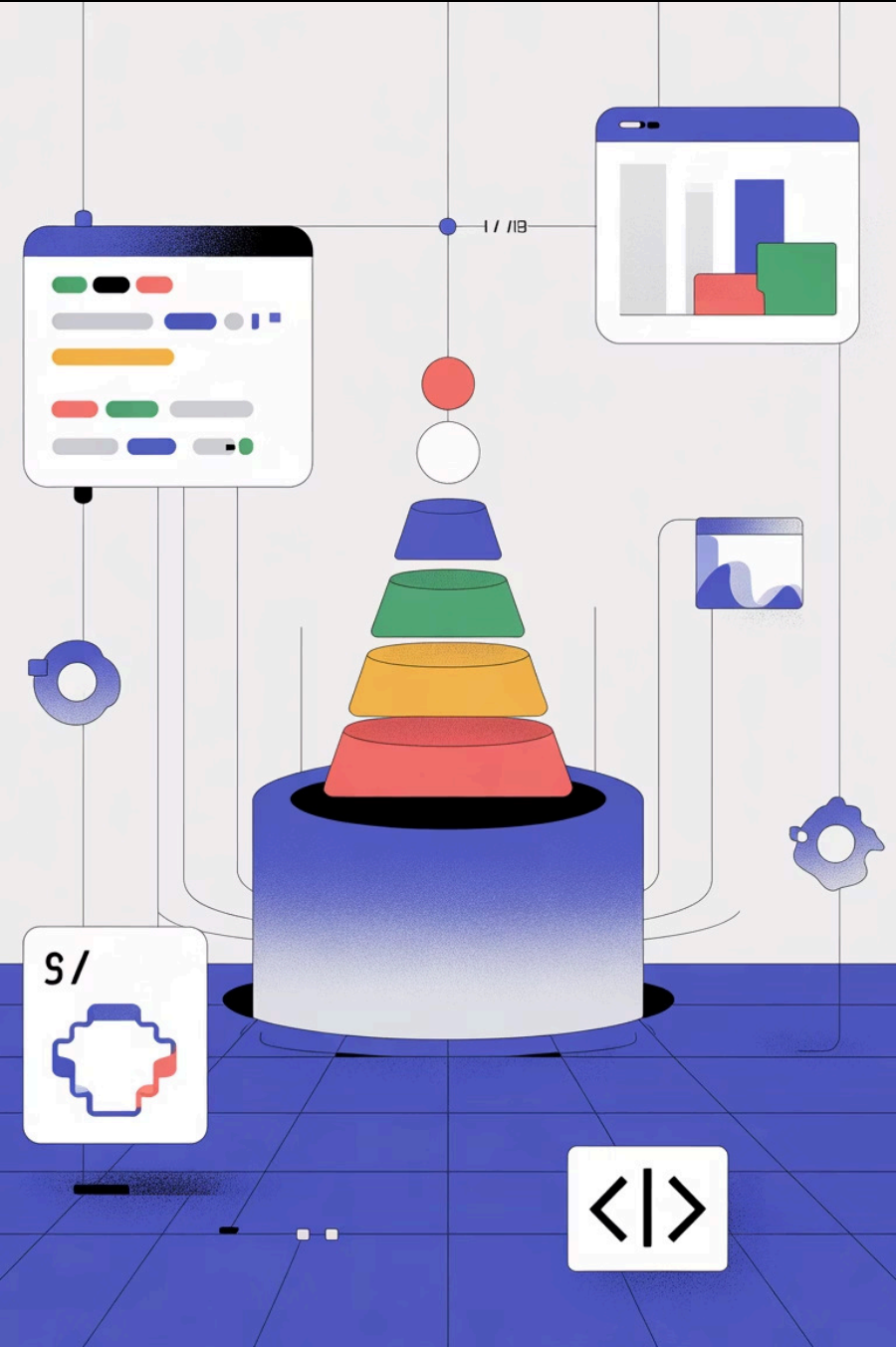
```
def sumaCola(n, acum=0):  
    if n == 0:  
        return acum  
    # Llamada recursiva es lo último  
    return sumaCola(n - 1, acum + n)  
  
print(sumaCola(5)) # 15
```

El acumulador `acum` lleva el resultado parcial. La llamada recursiva es la última operación; no hay cálculos pendientes.

Tabla comparativa: tradicional vs. cola

Aspecto	Recursión tradicional	Recursión de cola
Operaciones pendientes	Sí, después de la llamada recursiva	No, la llamada es lo último
Uso de pila (en Python)	Proporcional a la profundidad	Proporcional a la profundidad (sin TCO)
Optimización TCO	No aplicable	Posible en otros lenguajes, no en Python
Legibilidad	A menudo más intuitiva	Requiere acumuladores explícitos
Conversión a iteración	Más compleja	Más directa

En Python, la recursión de cola es principalmente un **patrón de diseño** que facilita la transformación a bucles iterativos, no una optimización automática.



Patrón: uso de acumuladores

El acumulador es un parámetro adicional que transporta el resultado parcial. En cada llamada recursiva, se actualiza el acumulador hasta alcanzar el caso base, donde se retorna directamente.

```
def fibonacciCola(n, a=0, b=1):  
    """Calcula el n-ésimo número de Fibonacci con recursión de cola."""  
    if n == 0:  
        return a  
    if n == 1:  
        return b  
    # Acumuladores a, b se actualizan en cada paso  
    return fibonacciCola(n - 1, b, a + b)  
  
# Prueba  
for i in range(10):  
    print(fibonacciCola(i), end=' ')  
# Salida: 0 1 1 2 3 5 8 13 21 34
```

Los parámetros `a` y `b` mantienen los dos últimos valores de la secuencia, eliminando la necesidad de cálculos posteriores a la llamada.

Transformación paso a paso



1. Identifica el caso base

Determina cuándo la función debe retornar sin recursión. Este no cambia.



2. Introduce acumuladores

Agrega parámetros para almacenar resultados parciales. Inicializa con valores neutros.



3. Mueve cálculos antes

Realiza las operaciones **antes** de la llamada recursiva, actualizando los acumuladores.



4. Retorna la llamada directamente

La llamada recursiva debe ser la última expresión, sin operaciones posteriores.



5. Verifica corrección

Prueba con casos pequeños y valida que el comportamiento sea idéntico.

Ejemplo: longitud de lista

Versión tradicional

```
def longitud_trad(lst):  
    if not lst:  
        return 0  
    # 1 + se calcula DESPUÉS  
    return 1 + longitud_trad(lst[1:])  
  
print(longitud_trad([1,2,3,4]))  
# Salida: 4
```

Versión de cola

```
def longitudCola(lst, acum=0):  
    if not lst:  
        return acum  
    # Incremento ANTES de recurrir  
    return longitudCola(lst[1:], acum+1)  
  
print(longitudCola([1,2,3,4]))  
# Salida: 4
```

Nota: `lst[1:]` crea una copia de la lista en cada llamada, lo cual es ineficiente. En práctica, usaríamos índices o iteración.

Implicaciones prácticas en Python

Sin optimización automática

La recursión de cola en Python consume la misma cantidad de memoria que la recursión tradicional. No hay beneficio de rendimiento directo.

Facilita conversión iterativa

Las funciones con recursión de cola son mecánicamente traducibles a bucles `while`, eliminando el riesgo de `RecursionError`.

Claridad algorítmica

Escribir funciones en forma de cola puede clarificar la lógica del algoritmo, haciendo explícito el estado en cada paso.

En resumen: la recursión de cola en Python es una **técnica de diseño**, no una optimización de rendimiento.

Conversión a bucle iterativo

```
def sumaCola(n, acum=0):  
    if n == 0:  
        return acum  
    return sumaCola(n - 1, acum + n)  
  
# Versión iterativa equivalente  
def suma_iterativa(n):  
    acum = 0  
    while n > 0:  
        acum = acum + n  
        n = n - 1  
    return acum  
  
print(suma_iterativa(5)) # 15
```

La transformación es directa: los parámetros se convierten en variables, el caso base se convierte en condición del bucle, y la actualización de acumuladores ocurre en el cuerpo del bucle.

📌 **Regla práctica:** Si tu función usa recursión de cola, casi siempre puede reescribirse como un bucle `while` más eficiente en Python.

Ejercicio #1: potencia con recursión de cola

Enunciado

Implementa una función `potenciaCola(base, exp, acum=1)` que calcule $base^{exp}$ usando recursión de cola.

Ejemplo:

```
potenciaCola(2, 5) # 32  
potenciaCola(3, 4) # 81
```

Restricciones:

- Usar recursión de cola (llamada como última operación)
- El exponente es entero no negativo
- Incluir caso base cuando `exp == 0`

Solución disponible al final de la presentación.

Pistas

1. ¿Qué valor debe tener el acumulador cuando `exp = 0`?
2. En cada paso, multiplica el acumulador por la base y decrementa el exponente.
3. Verifica que no haya operaciones después del `return` recursivo.